Guía de Actividades
Práctico-Experimentales Nro. 003

1. 1. Datos Generales

Integrantes	• Isaac Cartuche • Emilio Jaramillo • Isabel Morocho • Alexander Sanchez
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de de 2026 09 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. 2. Objetivo(s) de la Práctica:

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.
- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.
- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y

prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. 3. Materiales y reactivos:

Guia practica

Lista de ejercicios

4. 4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. 5. Procedimiento / Metodología

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

Situación:

Un estacionamiento dispone de 4 espacios. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

Procedimiento

1. Identifique el recurso compartido.
2. Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.
3. Cree un semáforo contador inicializado en 4.
4. Cree un proceso vehiculo(id).
5. El vehículo deberá solicitar un espacio antes de ingresar.
6. Si no existen espacios disponibles, deberá permanecer bloqueado.
7. Una vez que obtenga un espacio, deberá simular su permanencia.
8. Al salir, deberá liberar el espacio mediante signal().
9. Cree 10 procesos vehículo.
10. Analice qué sucede cuando llegan más de 4 vehículos simultáneamente.

Pseudocódigo

SEMÁFORO espacios = 4

PROCEDIMIENTO vehiculo(id)

```
    escribir("Vehículo ", id, " llega al estacionamiento")

    wait(espacios)

    escribir("Vehículo ", id, " ingresa al estacionamiento")

    estacionar(id)

    esperar_tiempo()

    escribir("Vehículo ", id, " sale del estacionamiento")

    signal(espacios)

FIN PROCEDIMIENTO


INICIO

    PARA i = 1 HASTA 10 HACER
        crear_proceso(vehiculo, i)
    FIN PARA

FIN
```

Pregunta de análisis:
¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

Ejercicio 2. Cajero automático compartido — Exclusión mutua
Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.
Situación:
Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura.
Procedimiento

1. Identifique la variable compartida.
2. Determine qué operaciones pueden ejecutarse simultáneamente.
3. Cree un mutex para proteger el saldo.
4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex.

5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.
7. Cree los tres procesos.
8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.
10. Compare el comportamiento con y sin exclusión mutua.

Pseudocódigo

```
saldo = 1000

MUTEX mutex_saldo = 1


PROCEDIMIENTO retirar(id, cantidad)

   lock(mutex_saldo)

   escribir("Proceso ", id, " intenta retirar $", cantidad)

   SI saldo >= cantidad ENTONCES

      saldo = saldo - cantidad

      escribir("Retiro realizado")
      escribir("Saldo actual: $", saldo)

   SINO

      escribir("Saldo insuficiente")

   FIN SI

   unlock(mutex_saldo)

FIN PROCEDIMIENTO


PROCEDIMIENTO depositar(id, cantidad)

   lock(mutex_saldo)

   escribir("Proceso ", id, " deposita $", cantidad)

   saldo = saldo + cantidad
```

```
    escribir("Saldo actual: $", saldo)

    unlock(mutex_saldo)

FIN PROCEDIMIENTO


INICIO

    crear_proceso(retirar, 1, 300)

    crear_proceso(retirar, 2, 250)
    crear_proceso(depositar, 3, 200)
FIN
```

Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:

Una fábrica tiene una línea de producción con capacidad para almacenar 5 piezas temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

Procedimiento

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.
5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.

14. Debe retirar la pieza.
15. Debe liberar el mutex.
16. Debe indicar que existe un espacio disponible.

Pseudocódigo

```
COLA almacen
CAPACIDAD = 5

SEMÁFORO espacios = 5
SEMÁFORO elementos = 0

MUTEX mutex_almacen = 1


PROCEDIMIENTO productor(id)

   REPETIR 5 VECES

      pieza = fabricar_pieza()

      wait(espacios)

      lock(mutex_almacen)

      insertar(almacen, pieza)

      escribir("Productor ", id,
            " agregó pieza ", pieza)

      unlock(mutex_almacen)

      signal(elementos)

   FIN REPETIR

FIN PROCEDIMIENTO


PROCEDIMIENTO consumidor(id)

   REPETIR 5 VECES

      wait(elementos)
```

```
        lock(mutex_almacen)

        pieza = extraer(almacen)

        escribir("Consumidor ", id,
              " retiró pieza ", pieza)

        unlock(mutex_almacen)

        signal(espacios)

        ensamblar(pieza)

    FIN REPETIR

FIN PROCEDIMIENTO


INICIO

    PARA i = 1 HASTA 2 HACER
        crear_proceso(productor, i)
    FIN PARA

    PARA i = 1 HASTA 2 HACER
        crear_proceso(consumidor, i)
    FIN PARA

FIN
```

Pregunta de análisis:

¿Qué sucede si espacios se inicializa en 0? ¿Qué sucede si elementos se inicializa en 5?

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación:

Un puente permite circular en ambos sentidos, pero solamente puede existir una dirección de circulación activa a la vez.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Procedimiento

1. Identifique el recurso compartido.
2. Defina una variable que indique la dirección actual.
3. Mantenga un contador de vehículos que están cruzando.
4. Proteja las variables mediante un mutex.
5. Un vehículo deberá verificar la dirección antes de entrar.

6. Si existen vehículos en dirección contraria, deberá esperar.
7. Si puede entrar, incrementará el contador correspondiente.
8. Después de cruzar, disminuirá el contador.
9. Cuando no queden vehículos, el puente podrá cambiar de dirección.
10. Analice el problema de inanición.

Pseudocódigo

```
MUTEX mutex = 1
direccion = NINGUNA
vehiculos_norte = 0
vehiculos_sur = 0


PROCEDIMIENTO vehiculo_norte(id)

   lock(mutex)

   MIENTRAS direccion == SUR HACER
      esperar()
   FIN MIENTRAS

   direccion = NORTE

   vehiculos_norte = vehiculos_norte + 1

   unlock(mutex)


   escribir("Vehículo ", id,
         " cruza hacia el NORTE")

   cruzar_puente(id)


   lock(mutex)

   vehiculos_norte = vehiculos_norte - 1
```

```
  SI vehiculos_norte == 0 ENTONCES
    direccion = NINGUNA
  FIN SI

  unlock(mutex)

FIN PROCEDIMIENTO
```

El estudiante deberá completar el procedimiento:
PROCEDIMIENTO vehiculo_sur(id)
siguiendo la misma lógica.
Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

Ejercicio 5. Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:

Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente:
DISEÑO → REVISIÓN → IMPRESIÓN
No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Procedimiento

1. Identifique los procesos participantes.
2. Cree un semáforo para cada etapa.
3. Permita inicialmente que el proceso de diseño pueda ejecutarse.
4. Diseño deberá habilitar a revisión.
5. Revisión deberá habilitar a impresión.
6. Impresión deberá habilitar nuevamente a diseño.
7. Repita el ciclo para 3 documentos.
8. Verifique que el orden siempre sea correcto.
9. Analice qué ocurre si se elimina uno de los signal().

Pseudocódigo

```
SEMÁFORO turno_diseno = 1
SEMÁFORO turno_revision = 0
SEMÁFORO turno_impresion = 0


PROCEDIMIENTO diseno()
```

```
    REPETIR 3 VECES
        wait(turno_diseno)
        escribir("Diseño prepara documento")
        crear_documento()
        signal(turno_revision)
    FIN REPETIR
FIN PROCEDIMIENTO

PROCEDIMIENTO revision()
    REPETIR 3 VECES
        wait(turno_revision)
        escribir("Revisión verifica documento")
        revisar_documento()
        signal(turno_impresion)
    FIN REPETIR
FIN PROCEDIMIENTO

PROCEDIMIENTO impresion()
    REPETIR 3 VECES
        wait(turno_impresion)
        escribir("Impresión imprime documento")
        imprimir_documento()
        signal(turno_diseno)
```

```
    FIN REPETIR

FIN PROCEDIMIENTO


INICIO

    crear_proceso(diseno)

    crear_proceso(revision)

    crear_proceso(impresion)

FIN
```

6. Resultados esperados:

- **Comprobación de las distintas formas de cómo los hilos se comunican.**
- **Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes**
- **El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la práctica**

Links de portafolios den Github de los estudiantes:
- Isaac Cartuche: https://github.com/IsaacCartuche/Sistemas_Distribuidos.git
- Isabel Morocho: https://github.com/IsaKatMM/Portafolio-S.Distribuidos
- Alexander Sánchez:
https://github.com/AlexanderSanchezCS/Portafolio-Sistemas-Distribuidos
- Emilio Jaramillo: https://github.com/emilioj04/APE03_SD

Ejercicio	Resultado Esperado	Criterio de Verificación
1. Estacionamiento	Vehículos dentro ≤ 4	Verificar que nunca ingresen más de 4 simultáneamente
2. Cajero	Saldo final = $650	Ejecutar varias veces y comprobar que el saldo no varíe
3. Productor-Consumidor	Todos los productos producidos son consumidos	Verificar que el almacén nunca supere 5 ni quede con inconsistencias
4. Puente	Solo una dirección activa a la vez	Comprobar que nunca crucen vehículos en sentidos opuestos simultáneamente
5. Centro de Impresión	Diseño → Revisión → Impresión	Verificar que ningún proceso avance a la siguiente etapa antes de tiempo

RESULTADOS - EJERCICIO 1

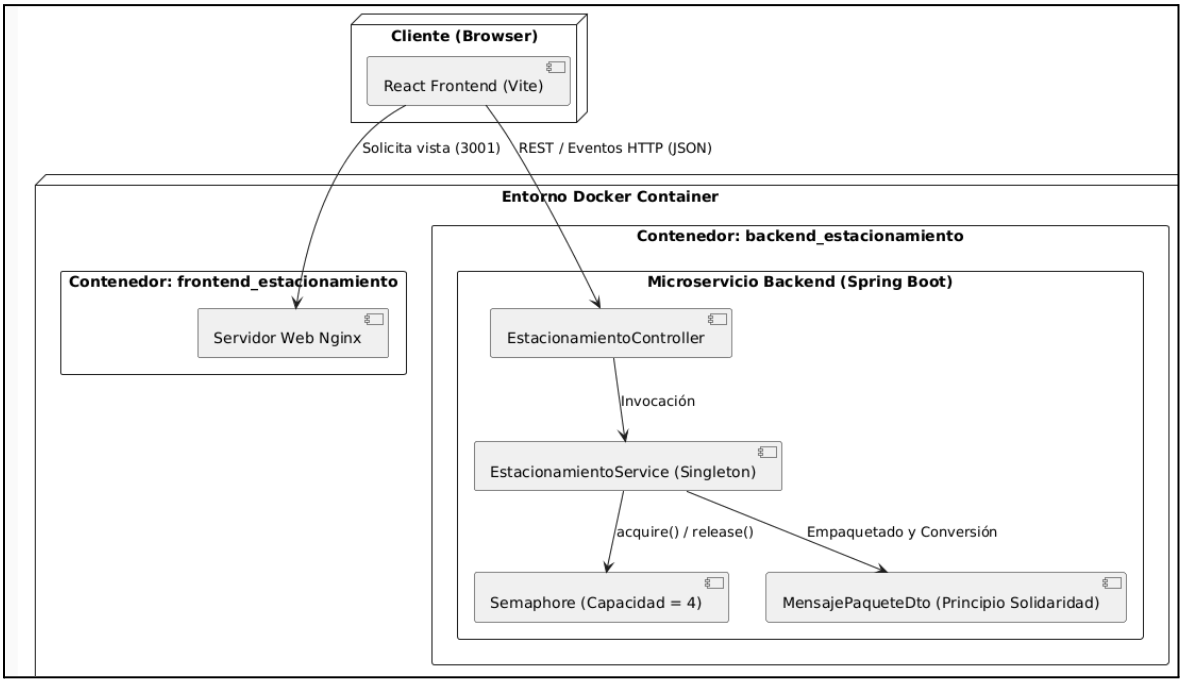
Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

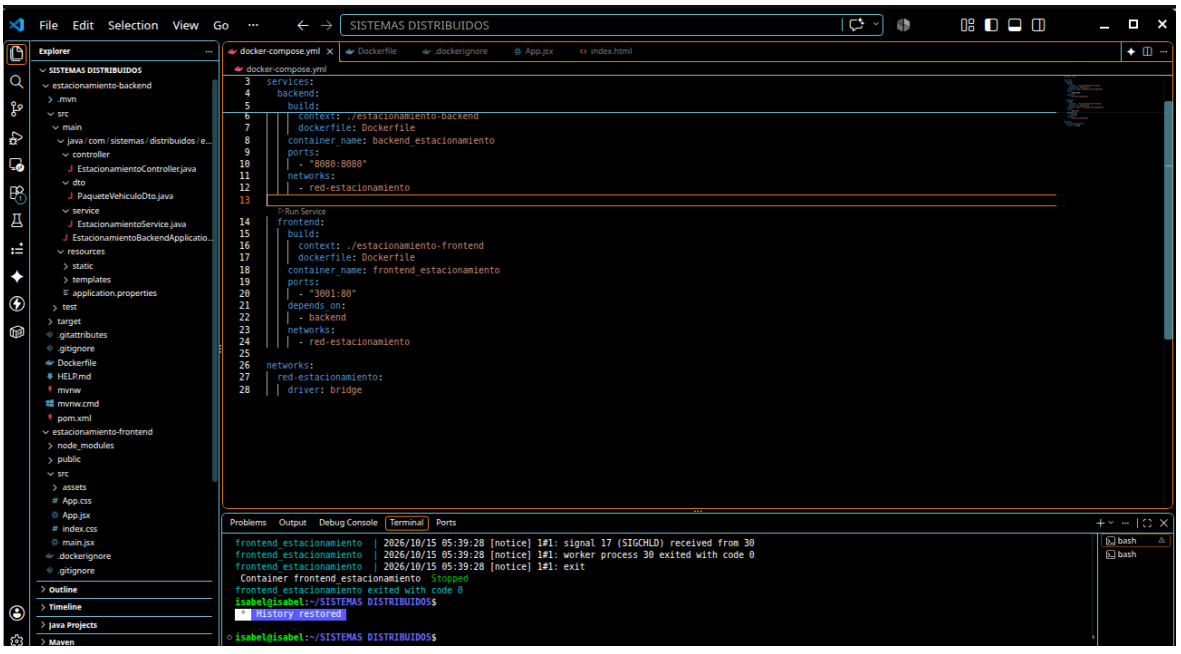
Situación:

Un estacionamiento dispone de 4 espacios. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

DIAGRAMA DE ARQUITECTURA



CARPETAS BACKEND - FRONTEND



SISTEMAS DISTRIBUIDOS/

```
|----- estacionamiento-backend/
|----- pom.xml
|----- mvnw.cmd
|-----mvnw
|----- HELP.md
|----- Dockerfile
|-----target
|----- .mvn/
|----- src/
    |-----main
    |-----java/com/sistemas/distribuidos/estacionamiento
      |-----controller
        |--------- EstacionamientoController.java
      |----- dto
        |--------- PaqueteVehiculoDto.java
      |----- service
        |--------- EstacionamientoService.java
     |--------- EstacionamientoBackendAPlication.java
|----- estacionamiento-frontend/
   |----- node_modeles
   |----- public
   |-----src
       |----- assets
          |----- App.jsx
  |----- dockerignore
  |----- Dockerfile
  |----- eslint.config.js
  |----- index.html
  |----- package-lock.json
  |----- package.json
  |----- README.md
  |----- docker-compose.yml
```

La solución está organizada bajo una arquitectura de microservicios contenarizados con Docker, dividida en un backend desarrollado en Spring Boot y un frontend en React (Vite)

pom.xml: Contiene las dependencias necesarias para la ejecución del backend

estacionamiento-backend > pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.8</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.sistemas.distribuidos</groupId>
    <artifactId>estacionamiento-backend</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name/>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
```

Dockerfile - Backend

estacionamiento-backend > Dockerfile > ...

```dockerfile
# Etapa 1: Compilación de la aplicación Java
FROM eclipse-temurin:17-jdk-alpine AS builder
WORKDIR /app

COPY .mvn/ .mvn
COPY mvnw pom.xml ./
RUN chmod +x mvnw
RUN ./mvnw dependency:go-offline

COPY src ./src
RUN ./mvnw clean package -DskipTests

# Etapa 2: Imagen final liviana
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

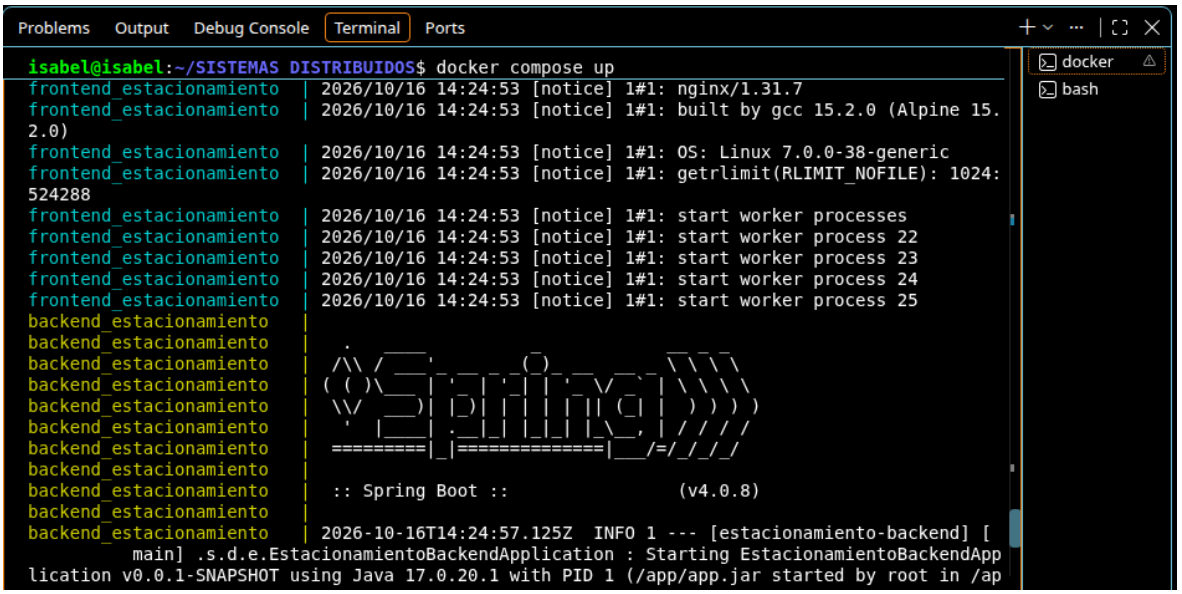
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

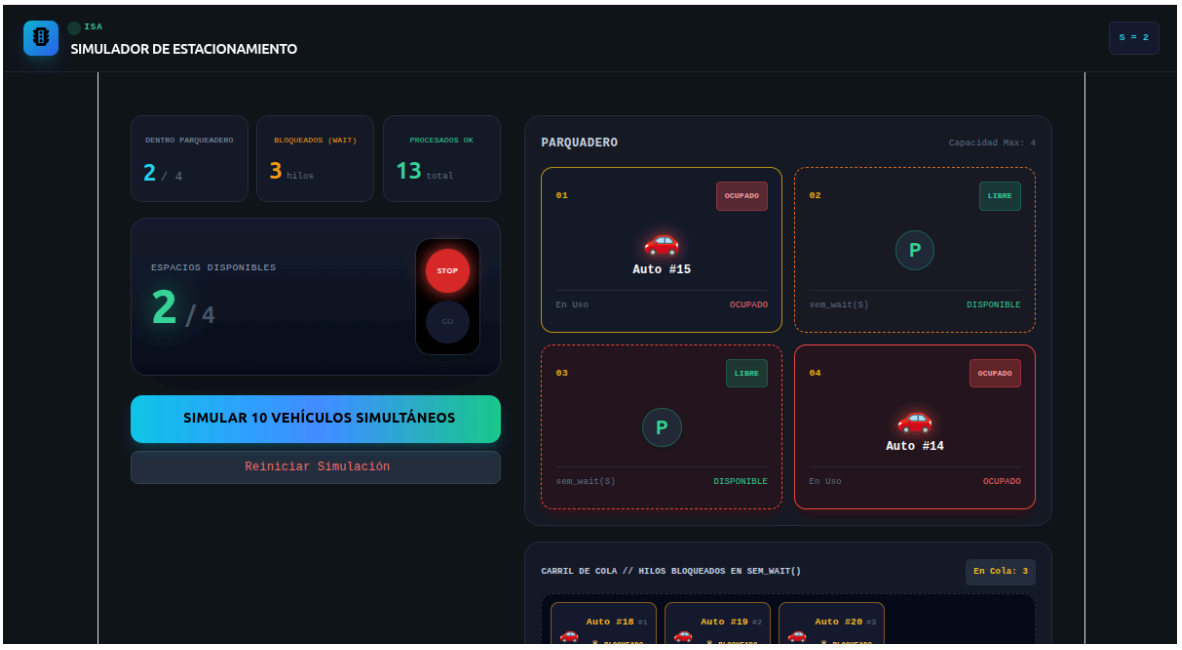
Inicialmente se construyó únicamente el Dockerfile del backend para aislar la lógica del negocio (Semaphore(4, true)), asegurando que la API REST funcionara correctamente de manera independiente. Posteriormente, al integrar la interfaz de usuario en React, se evolucionó la arquitectura hacia un modelo de orquestación completa con Docker Compose (docker-compose.yml). Lo cual me permitió automatizar el empaquetado, la creación de redes aisladas (red-estacionamiento) y la gestión de dependencias (depends_on), logrando desplegar todo el sistema distribuido (Backend y Frontend) mediante un único comando unificado (docker compose up), lo que garantiza la repetibilidad del entorno de producción.

Ejecución del backend - frontend

Se realizó mediante el comando docker compose up



Frontend visual - simulación



Metodología: La metodología aplicada fue incremental, ya que se empezo resolviendo el problema lógico de concurrencia mediante un semáforo contador Singleton en Spring Boot. Luego, se validó el Backend de forma aislada en su propio contenedor Docker. Finalmente, se construyó la interfaz en React y se integró ambos microservicios mediante Docker Compose, logrando un despliegue orquestado, en una sola red aislada

Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

- **Si el semáforo se inicializa en 1:** El estacionamiento funcionará como una sección crítica de exclusión mutua estricta (Mutex), permitiendo solo 1 vehículo a la vez.
- **Si se inicializa en 10:** Entrarán los 10 vehículos simultáneamente sin

bloquear a ninguno, perdiendo la restricción de capacidad máxima del estacionamiento

RESULTADOS - EJERCICIO 2

Cajero automático compartido — Exclusión mutua

Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

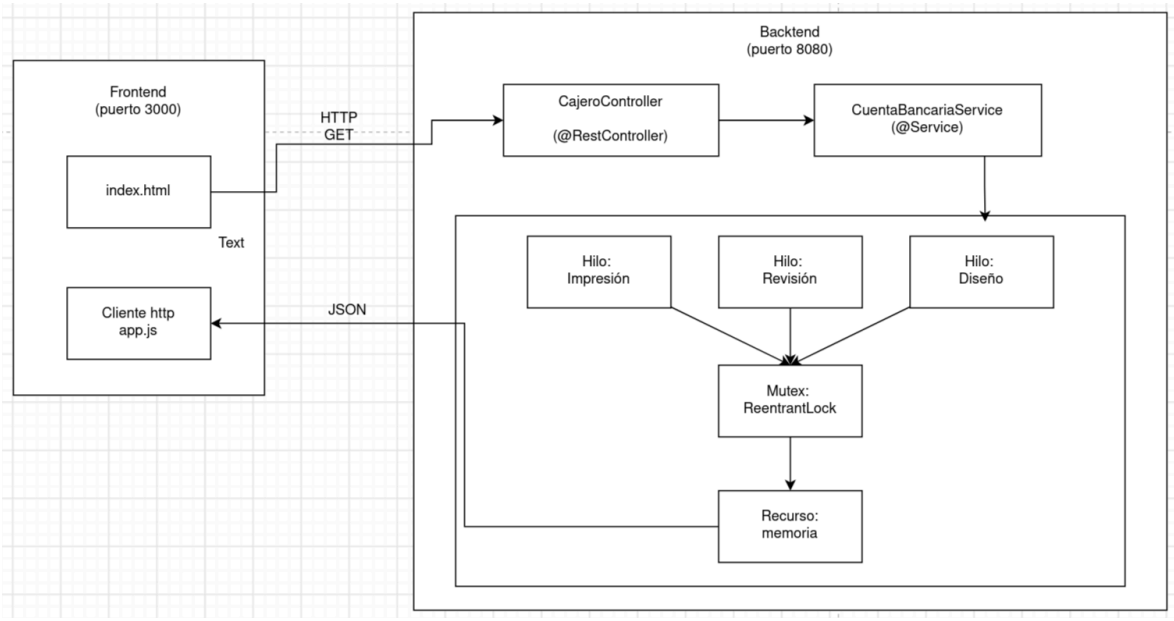
- **Proceso 1:** retiro de $300.
- **Proceso 2:** retiro de $250.
- **Proceso 3:** depósito de $200.

Se determina el uso de una estructura similar a microservicios, siguiendo con el uso de la tecnologías seleccionadas se tiene por parte del BackEnd el uso de Java con Spring Boot, mientras que por el lado del FrontEnd se hara uso de un servidor HTTP de python para su funcionamiento sin necesidad de dependencias.

Estructura de Archivos:

```
proyecto-distribuidos/
├── backend-cajero/
│   └── src/
│       └── main/
│           └── java/
│               └── com/
│                   └── practica/
│                       └── cajero/
│                           ├── CajeroApplication.java
│                           ├── controller/
│                           │   └── CajeroController.java
│                           └── service/
│                               └── CuentaBancariaService.java
└── frontend-cajero/
    ├── index.html
    ├── style.css
    └── app.js
```

Diagrama de arquitectura:



Para la ejecución del Backend se hizo uso de un contenedor en Docker, facilitando la ejecución y evitando la instalación de dependencias locales en un solo host, para ello se prepararon 2 archivos fundamentales para la creación de la imagen de Docker correspondiente.:

pom.xml : Este archivo contiene las dependencias e instalaciones necesarias para el funcionamiento del backend.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="[http://maven.apache.org/POM/4.0.0](http://maven.apache.org/POM/4.0.0)" xmlns:xsi="[http://www.w3.org/
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.2.4</version>
</parent>
<groupId>com.practica</groupId>
<artifactId>cajero</artifactId>
<version>0.0.1-SNAPSHOT</version>
<properties>
<java.version>17</java.version>
</properties>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Dockerfile : Este archivo identifica la estructura de archivos y genera la imagen únicamente del Backend.

```
FROM maven:3.9-eclipse-temurin-17 AS build (las
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

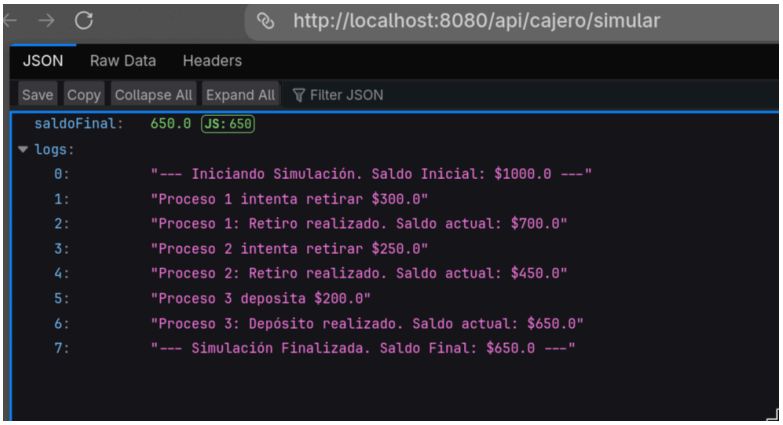
FROM eclipse-temurin:17-jre-alpine (last pushed
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

Para la ejecución del Backend se realizó la utilización del siguiente comando de especificación del puerto 8080.

docker run -p 8080:8080 cajero-backend



Dando así el funcionamiento del Backend de manera local:



Para el levantamiento del Frontend se utiliza únicamente un servidor local HTTP de python en el puerto 3000 con el siguiente comando.
python -m http.server 3000



Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

Al tratarse de varios procesos que buscan el acceso a un mismo espacio de memoria pueden ocurrir algunos escenarios en los que se sobreponga valores calculados por algún proceso, o incluso se corrompa el espacio de memoria. para evitar cosas como estas se utiliza un mutex permitiendo un mejor control al acceso a este recurso y así mejorando la integridad de la vida del recurso.

RESULTADOS - EJERCICIO 3

Fábrica de ensamblaje — Productor-Consumidor
Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:
Una fábrica tiene una línea de producción con capacidad para almacenar 5 piezas temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

Estructura de Archivos:

APE3_Alexander_Sanchez

```
├── backend-productor-consumidor
│   ├── almacen-service
│   │   ├── Dockerfile
│   │   ├── HELP.md
│   │   ├── mvnw
│   │   ├── mvnw.cmd
│   │   ├── pom.xml
│   │   └── src
│   │       ├── main
│   │       │   ├── java
│   │       │   │   └── ec
│   │       │   │       └── unl
│   │       │   │           └── distribuidos
│   │       │   │               └── almacen
│   │       │   │                   ├── AlmacenController.java
│   │       │   │                   ├── AlmacenServiceApplication.java
│   │       │   │                   └── BufferAcotado.java
│   │       │   └── resources
│   │       │       ├── application.properties
│   │       │       ├── static
│   │       │       └── templates
│   │       └── test
│   │           └── java
│   │               └── ec
│   │                   └── unl
│   │                       └── distribuidos
│   │                           └── almacen
│   │                               └── AlmacenServiceApplicationTests.java
│   ├── consumidor-service
│   │   ├── Dockerfile
│   │   ├── HELP.md
│   │   ├── mvnw
│   │   ├── mvnw.cmd
```

```
│   │   ├── pom.xml
│   │   └── src
│   │       ├── main
│   │       │   ├── java
│   │       │   │   └── ec
│   │       │   │       └── unl
│   │       │   │           └── distribuidos
│   │       │   │               └── consumidor
│   │       │   │                   ├── AlmacenClient.java
│   │       │   │                   ├── ConsumidorController.java
│   │       │   │                   └── ConsumidorServiceApplication.java
│   │       │   └── resources
│   │       │       ├── application.properties
│   │       │       ├── static
│   │       │       └── templates
│   │       └── test
│   │           └── java
│   │               └── ec
│   │                   └── unl
│   │                       └── distribuidos
│   │                           └── consumidor
│   │                               └── ConsumidorServiceApplicationTests.java
│   └── productor-service
│       ├── Dockerfile
│       ├── HELP.md
│       ├── mvnw
│       ├── mvnw.cmd
│       ├── pom.xml
│       └── src
│           ├── main
│           │   ├── java
│           │   │   └── ec
│           │   │       └── unl
│           │   │           └── distribuidos
│           │   │               └── productor
│           │   │                   ├── AlmacenClient.java
│           │   │                   ├── ProductorController.java
│           │   │                   └── ProductorServiceApplication.java
│           │   └── resources
│           │       ├── application.properties
│           │       ├── static
│           │       └── templates
│           └── test
│               └── java
│                   └── ec
│                       └── unl
│                           └── distribuidos
```

```
│                    └── productor
│                        └── ProductorServiceApplicationTests.java
├── docker-compose.yml
└── frontend-productor-consumidor
    └── index.html
```

Arquitectura de microservicios:

almacen-service: mantiene la cola compartida y la sincronización.

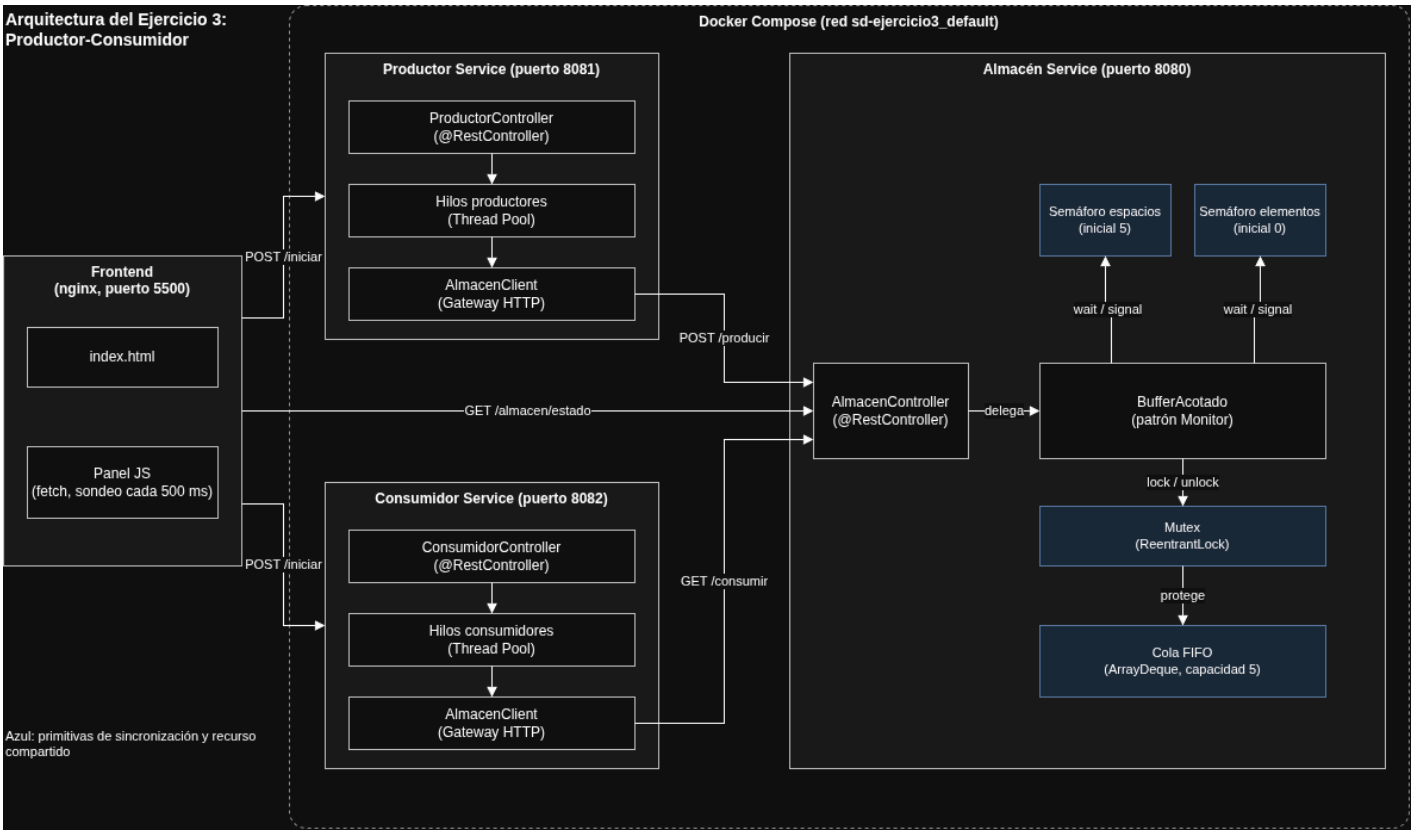
productor-service: crea hilos productores y solicita insertar piezas.

consumidor-service: crea hilos consumidores y solicita retirar piezas.

Frontend: permite iniciar a productores y consumidores, observar la cola y visualizar métricas.

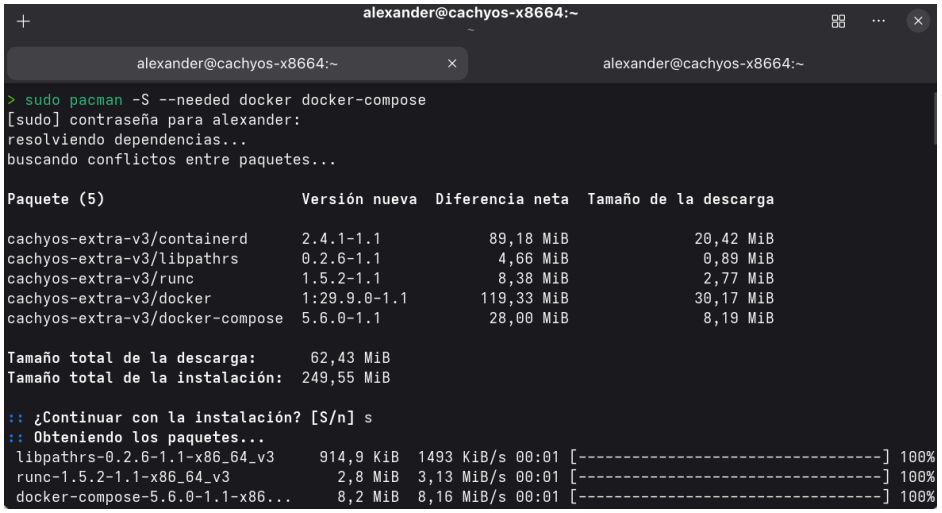
Nginx: sirve el frontend mediante Docker.

Diagrama-Arquitectura-Fábrica-Ensamblaje

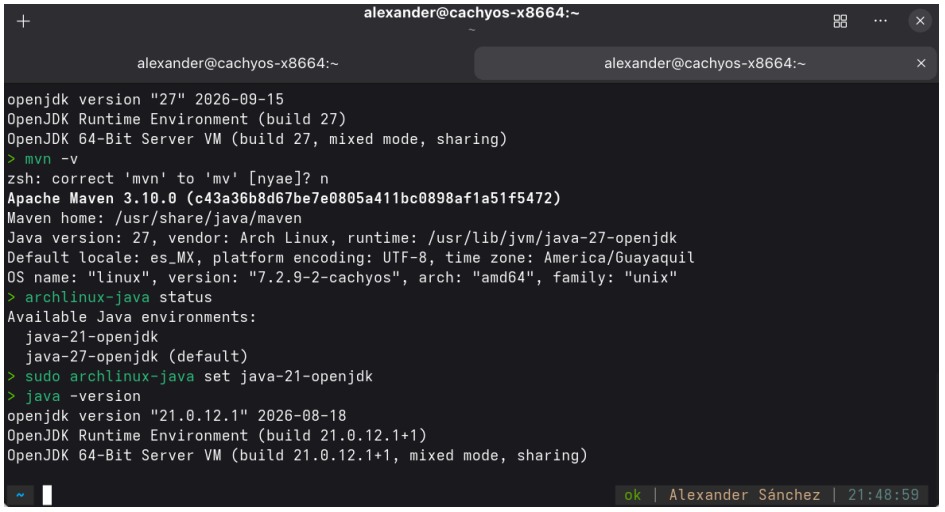


Desarrollo:

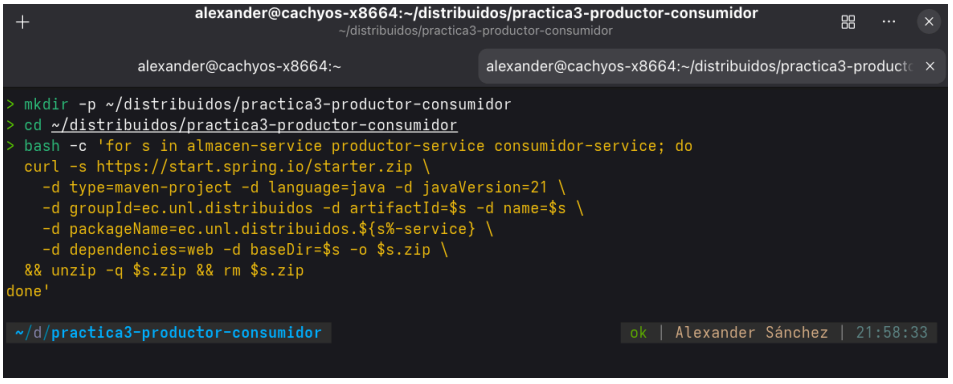
1. Instalación de Docker



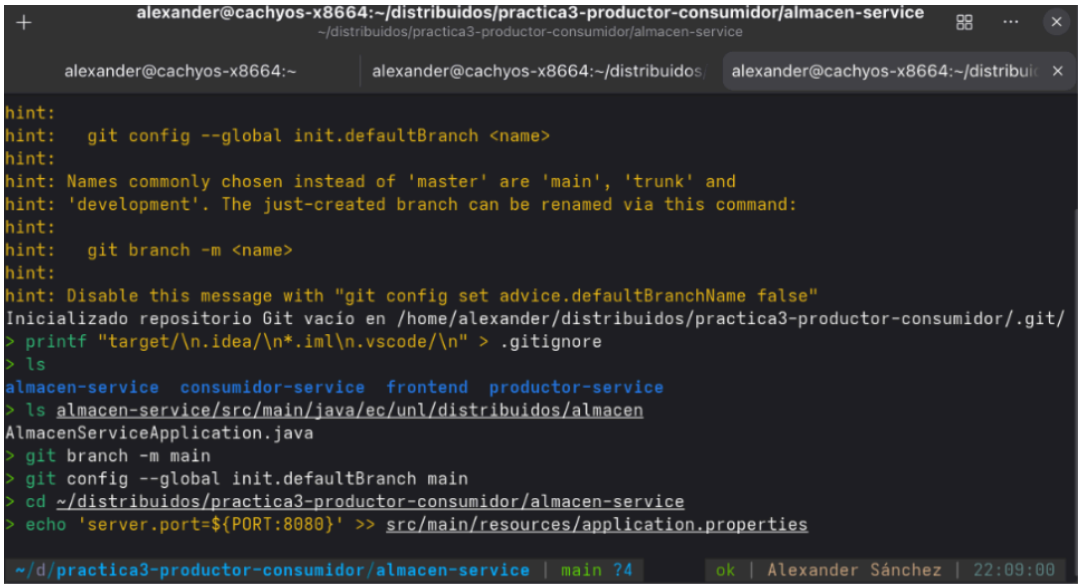
2. Herramientas (Java 21, Maven, git, unzip)



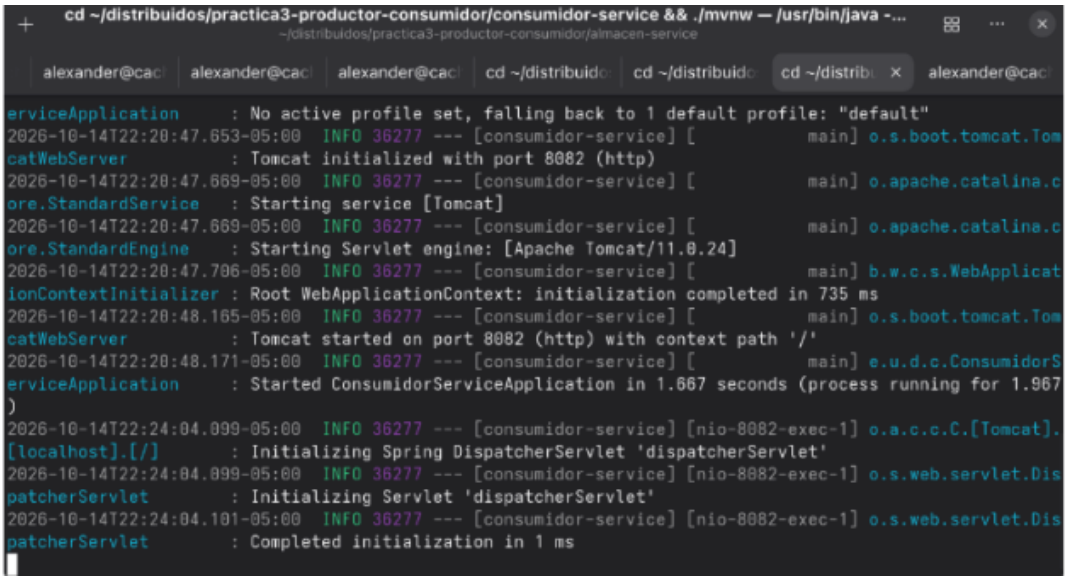
3. Carpeta del proyecto y los 3 microservicios Spring Boot



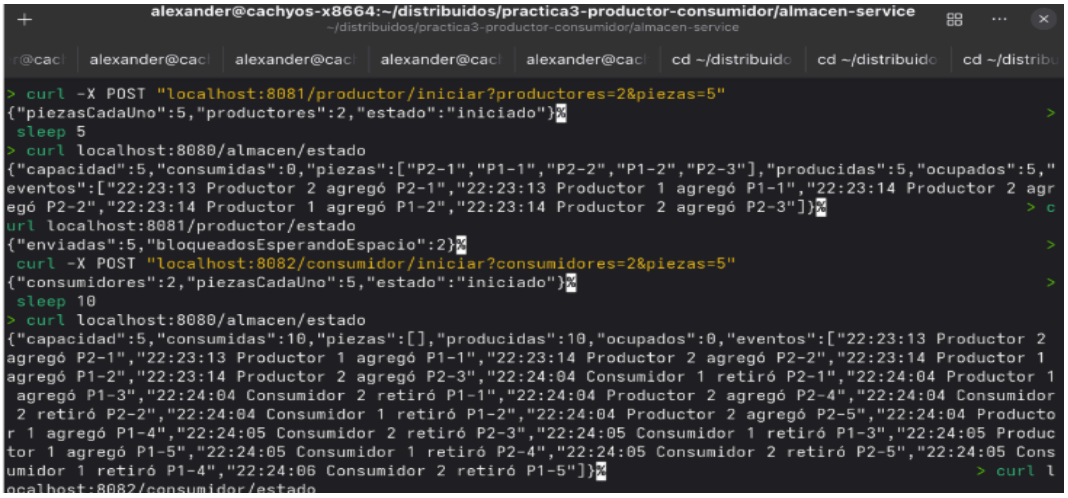
4. Puerto Configurable(Docker)



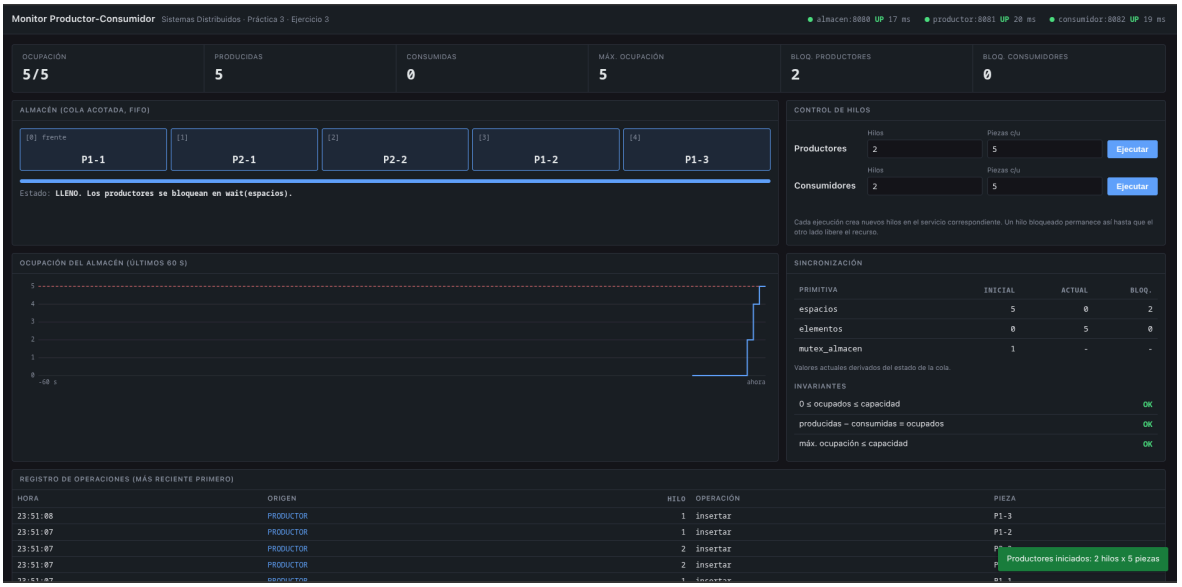
5. Terminales de servicios



6. Ejecución Terminal (Ejercicio)



7. Interfaz funcional



8. Registro de operaciones



Pregunta de análisis:

¿Qué sucede si espacios se inicializan en 0? ¿Qué sucede si elementos se inicializan en 5?

```
> F=backend-productor-consumidor/almacen-service/src/main/java/ec/unl/distribuidos/almacen/BufferAcotado.java
> git restore "$F"
> sed -i 's|new Semaphore(capacidad)|new Semaphore(0)|' "$F"
> grep -n "Semaphore" "$F"
8:import java.util.concurrent.Semaphore;
16:    private final Semaphore espacios;                    // espacios libres
17:    private final Semaphore elementos = new Semaphore(0);      // piezas disponibles
26:        this.espacios = new Semaphore(0);
```

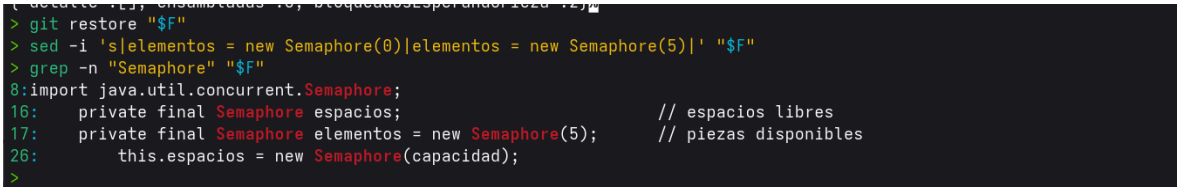
```
[+] up 2/2
 ✓ Image sd-almacen:1.0              Built                                    10.5s
 ✓ Container sd-ejercicio3-almacen-1 Started                                   1.4s
> docker compose restart productor consumidor
[+] restart 0/2
 ⠿ Container sd-ejercicio3-consumidor-1 Restarting                             1.3s
 ⠿ Container sd-ejercicio3-productor-1  Restarting                             1.3s
> curl -X POST "localhost:8081/productor/iniciar?productores=2&piezas=5"
{"estado":"iniciado","piezasCadaUno":5,"productores":2}%
> curl -X POST "localhost:8082/consumidor/iniciar?consumidores=2&piezas=5"
{"consumidores":2,"piezasCadaUno":5,"estado":"iniciado"}%
> sleep 5
> curl localhost:8080/almacen/estado
{"consumidas":0,"ocupados":0,"producidas":0,"maxOcupados":0,"capacidad":5,"eventos":[],"piezas":[]}%
> curl localhost:8081/productor/estado
{"bloqueadosEsperandoEspacio":2,"enviadas":0}%
> curl localhost:8082/consumidor/estado
{"detalle":[],"ensambladas":0,"bloqueadosEsperandoPieza":2}%
```

*Resultado con el semáforo espacios **inicializado en 0**: los dos productores y los dos consumidores quedan bloqueados.*

Con espacios en 0 y elementos en 0, el productor ejecuta wait(espacios) y se bloquea de inmediato, aunque el almacén esté vacío, porque no hay permisos disponibles.

El consumidor ejecuta wait(elementos) y también se bloquea, porque no hay

piezas. Para que el productor avance se necesita un signal(espacios), que solo emite un consumidor después de retirar una pieza. Para que el consumidor avance se necesita un signal(elementos), que solo emite un productor después de insertar. Cada grupo espera una señal que depende del otro, por lo que se produce un interbloqueo (deadlock). En la prueba se observó producidas: 0, consumidas: 0, dos productores bloqueados esperando espacio y dos consumidores bloqueados esperando pieza.





*Resultado con el semáforo elementos **inicializado en 5**: las solicitudes de consumo fallan con HTTP 500.*

Con elementos en 5, el semáforo afirma que hay cinco piezas disponibles cuando la cola está vacía. Los consumidores superan wait(elementos) sin bloquearse, adquieren el mutex e intentan extraer de una cola vacía, lo que lanza NoSuchElementException. El servicio responde con HTTP 500. En la prueba, las tres solicitudes de consumo devolvieron 500 y el estado quedó sin cambios (ocupados: 0, consumidas: 0).

Además, cada intento fallido consume un permiso de elementos (pasa de 5 a 2 tras tres intentos) sin que exista pieza alguna, y la excepción ocurre antes de signal(espacios). El semáforo queda desincronizado respecto a la cola, se rompe el invariante espacios + elementos = capacidad (aquí suma 7 en lugar de 5) y los consumos posteriores seguirán fallando hasta agotar los permisos falsos.

Conclusión:

Los semáforos deben inicializarse con el estado real del buffer: espacios en la capacidad (5), porque todos los lugares están libres, y elementos en 0, porque no hay piezas. Con esos valores se cumple en todo momento espacios + elementos = capacidad, el productor solo se bloquea cuando el almacén está lleno y el consumidor solo se bloquea cuando está vacío. Una inicialización incorrecta produce interbloqueo (caso 1) o inconsistencia entre el semáforo y la cola (caso 2).

RESULTADOS - EJERCICIO 4

Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación:

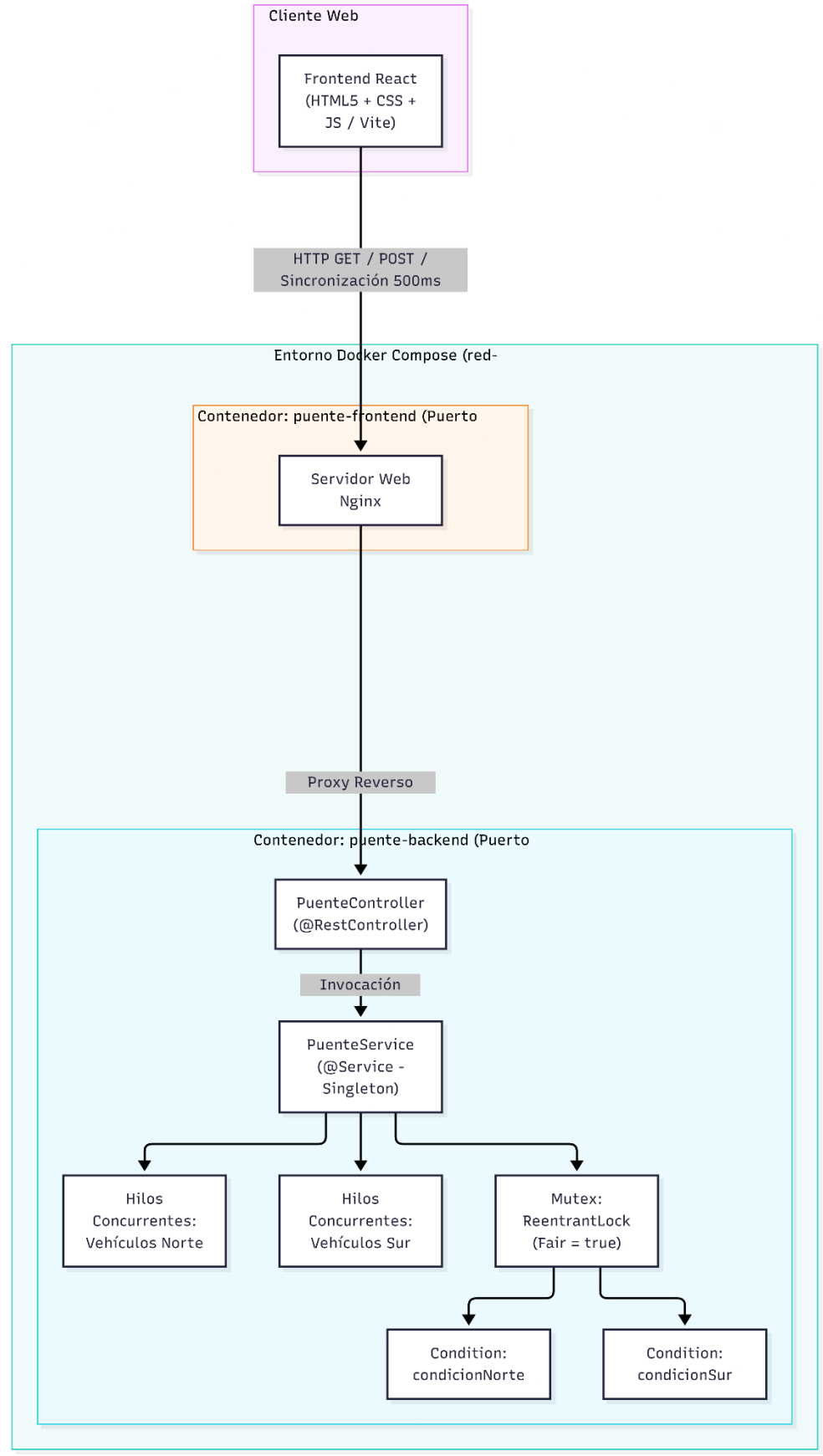
Un puente permite circular en ambos sentidos, pero solamente puede existir una dirección de circulación activa a la vez.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Estructura de archivos:

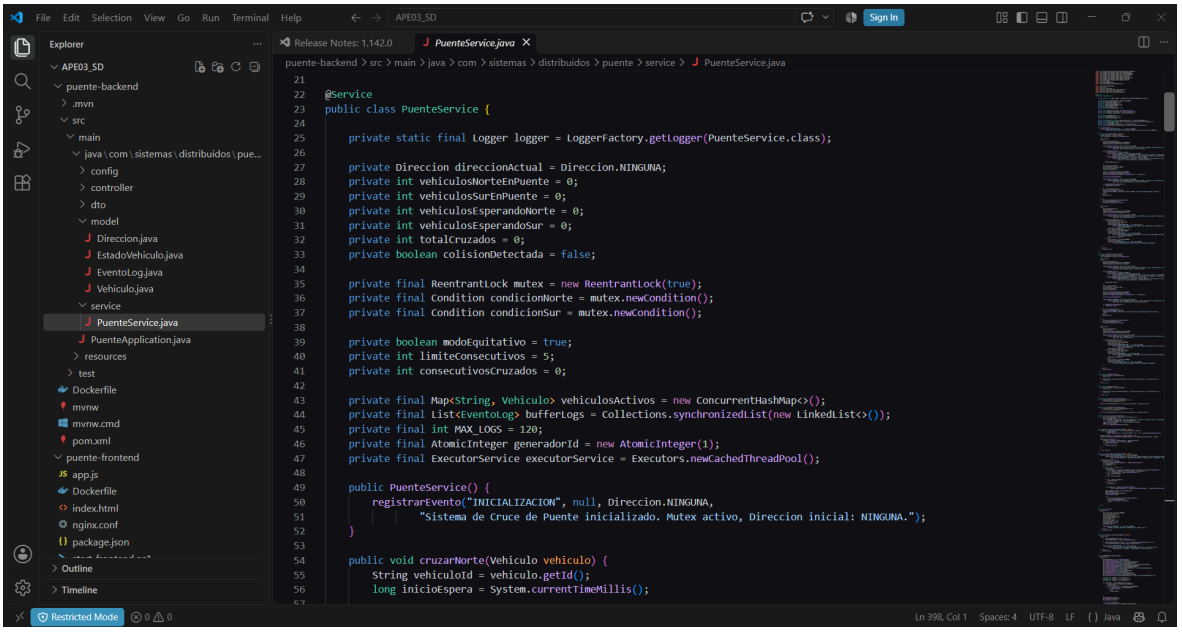
```
APE03_SD/
├── docker-compose.yml
├── README.md
├── puente-backend/
│   ├── Dockerfile
│   ├── pom.xml
│   ├── mvnw / mvnw.cmd
│   └── src/
│       └── main/
│           ├── java/com/sistemas/distribuidos/puente/
│           │   ├── PuenteApplication.java
│           │   ├── config/
│           │   │   └── WebCorsConfig.java
│           │   ├── controller/
│           │   │   └── PuenteController.java
│           │   ├── dto/
│           │   │   ├── EstadoPuenteDto.java
│           │   │   ├── SimulacionRequest.java
│           │   │   └── VehiculoRequest.java
│           │   ├── model/
│           │   │   ├── Direccion.java
│           │   │   ├── EstadoVehiculo.java
│           │   │   └── Vehiculo.java
│           │   └── service/
│           │       └── PuenteService.java
│           └── resources/
│               └── application.properties
└── puente-frontend/
    ├── Dockerfile
    ├── nginx.conf
    ├── index.html
    ├── style.css
    └── app.js
```

Diagrama de Arquitectura:


Cliente Web
Frontend React (HTML5 + CSS + JS / Vite)
HTTP GET / POST / Sincronización 500ms
Entorno Docker Compose (red-
Contenedor: puente-frontend (Puerto
Servidor Web Nginx
Proxy Reverso
Contenedor: puente-backend (Puerto
PuenteController (@RestController)
Invocación
PuenteService (@Service - Singleton)
Hilos Concurrentes: Vehículos Norte
Hilos Concurrentes: Vehículos Sur
Mutex: ReentrantLock (Fair = true)
Condition: condicionNorte
Condition: condicionSur

Implementación del Algoritmo y Mecanismo de Sincronización

La sincronización se resolvió en la capa de servicios mediante el patrón Monitor utilizando ReentrantLock(fair = true) y dos variables de condición (Condition), incorporando un contador de cruces consecutivos para evitar la inanición de la cola contraria.



```java
@Service
public class PuenteService {

    private static final Logger logger = LoggerFactory.getLogger(PuenteService.class);

    private Direccion direccionActual = Direccion.NINGUNA;
    private int vehiculosNorteEnPuente = 0;
    private int vehiculosSurEnPuente = 0;
    private int vehiculosEsperandoNorte = 0;
    private int vehiculosEsperandoSur = 0;
    private int totalCruzados = 0;
    private boolean colisionDetectada = false;

    private final ReentrantLock mutex = new ReentrantLock(true);
    private final Condition condicionNorte = mutex.newCondition();
    private final Condition condicionSur = mutex.newCondition();

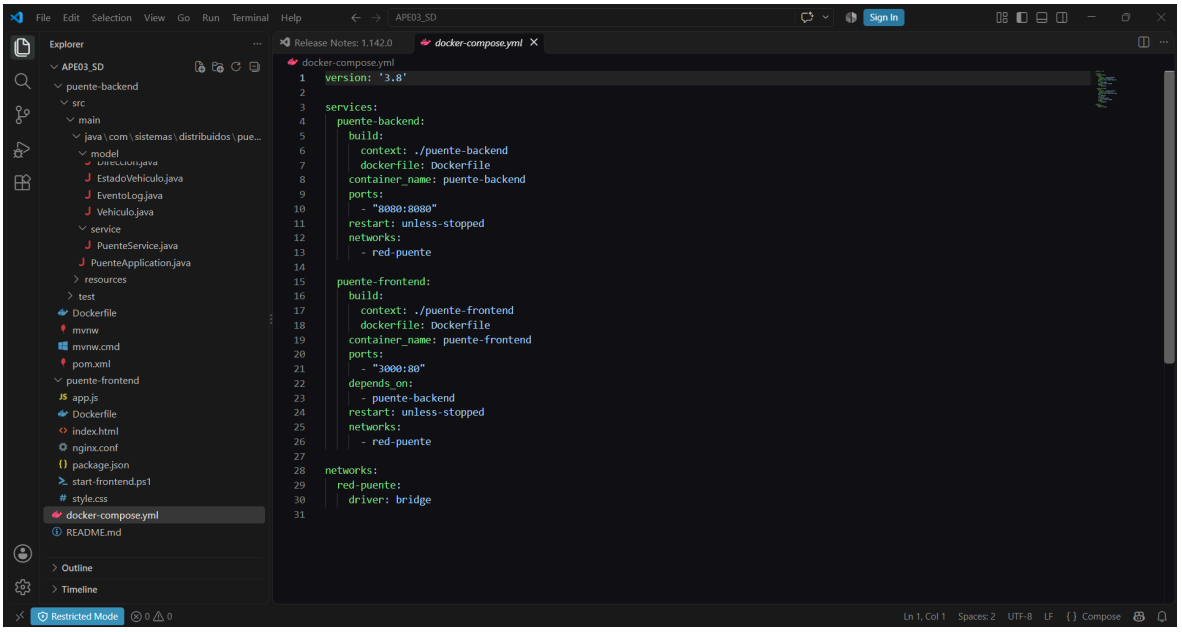
    private boolean modoEquitativo = true;
    private int limiteConsecutivos = 5;
    private int consecutivosCruzados = 0;

    private final Map<String, Vehiculo> vehiculosActivos = new ConcurrentHashMap<>();
    private final List<EventoLog> bufferLogs = Collections.synchronizedList(new LinkedList<>());
    private final int MAX_LOGS = 120;
    private final AtomicInteger generadorId = new AtomicInteger(1);
    private final ExecutorService executorService = Executors.newCachedThreadPool();

    public PuenteService() {
        registrarEvento("INICIALIZACION", null, Direccion.NINGUNA,
                "Sistema de Cruce de Puente inicializado. Mutex activo, Direccion inicial: NINGUNA.");
    }

    public void cruzarNorte(Vehiculo vehiculo) {
        String vehiculoId = vehiculo.getId();
        long inicioEspera = System.currentTimeMillis();
```

Archivo de Orquestación (docker-compose.yml)



```yaml
version: '3.8'

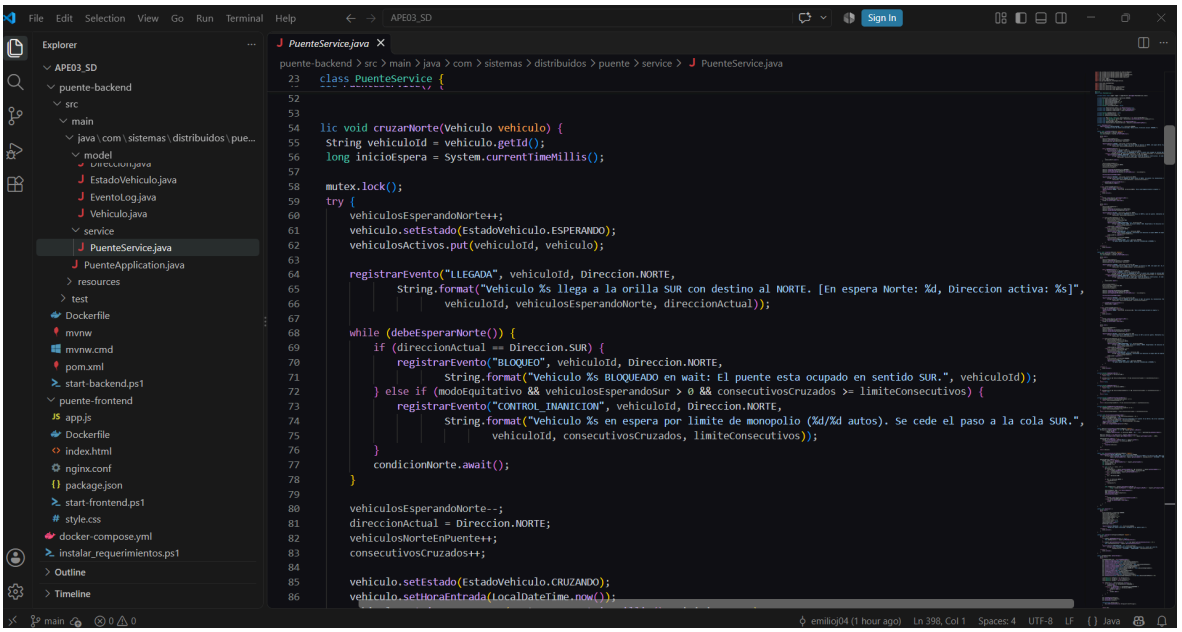
services:
  puente-backend:
    build:
      context: ./puente-backend
      dockerfile: Dockerfile
    container_name: puente-backend
    ports:
      - "8080:8080"
    restart: unless-stopped
    networks:
      - red-puente

  puente-frontend:
    build:
      context: ./puente-frontend
      dockerfile: Dockerfile
    container_name: puente-frontend
    ports:
      - "3000:80"
    depends_on:
      - puente-backend
    restart: unless-stopped
    networks:
      - red-puente

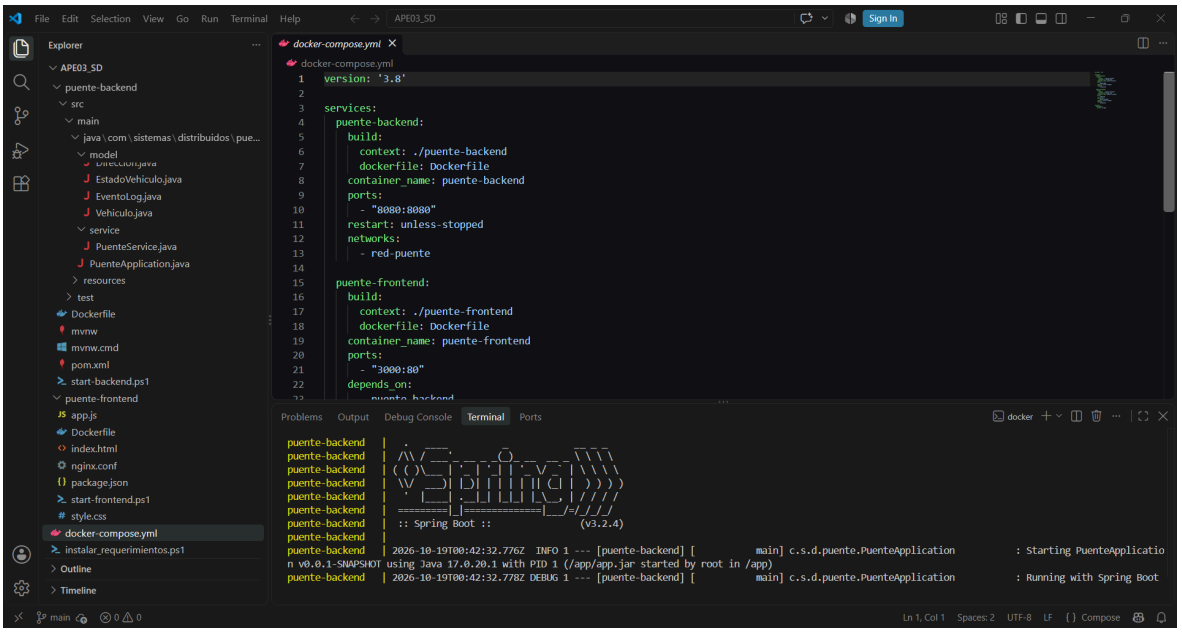
networks:
  red-puente:
    driver: bridge
```

Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

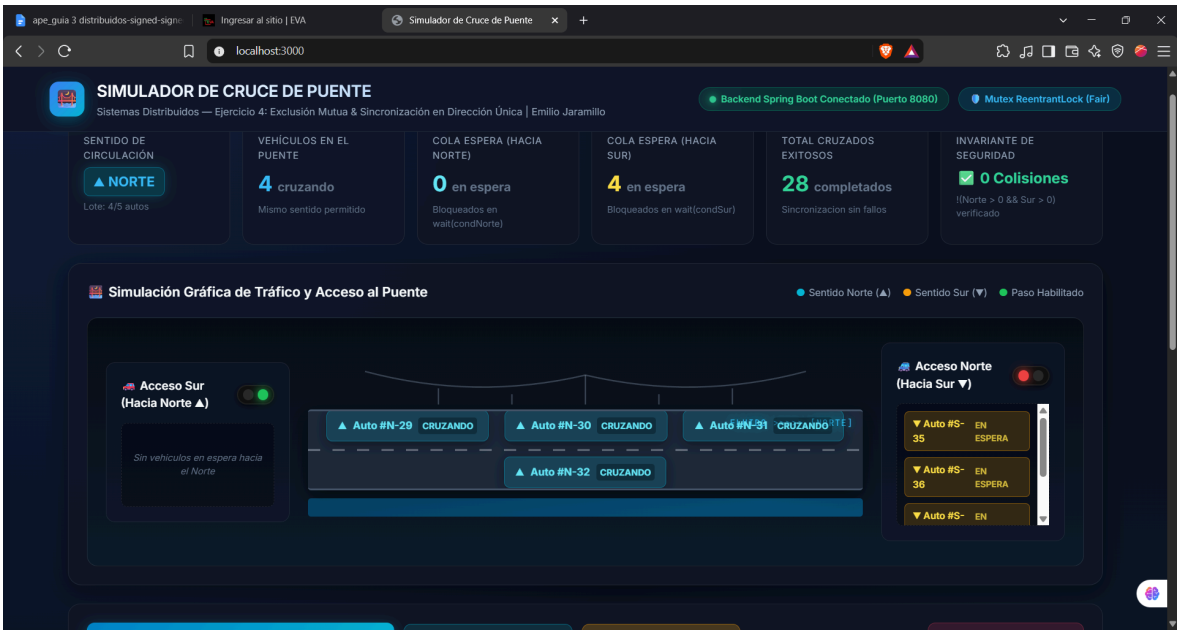
El reto se resuelve incorporando un límite de vehículos consecutivos (LIMITE_LOTE = 5) y verificando si hay vehículos esperando en la dirección contraria antes de permitir el ingreso de un nuevo auto.



Comando de Inicialización:
docker compose up --build



Resultados Visuales y Pruebas de Simulación en la Interfaz Web



RESULTADOS - EJERCICIO 5

Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:

Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente:

DISEÑO → REVISIÓN → IMPRESIÓN

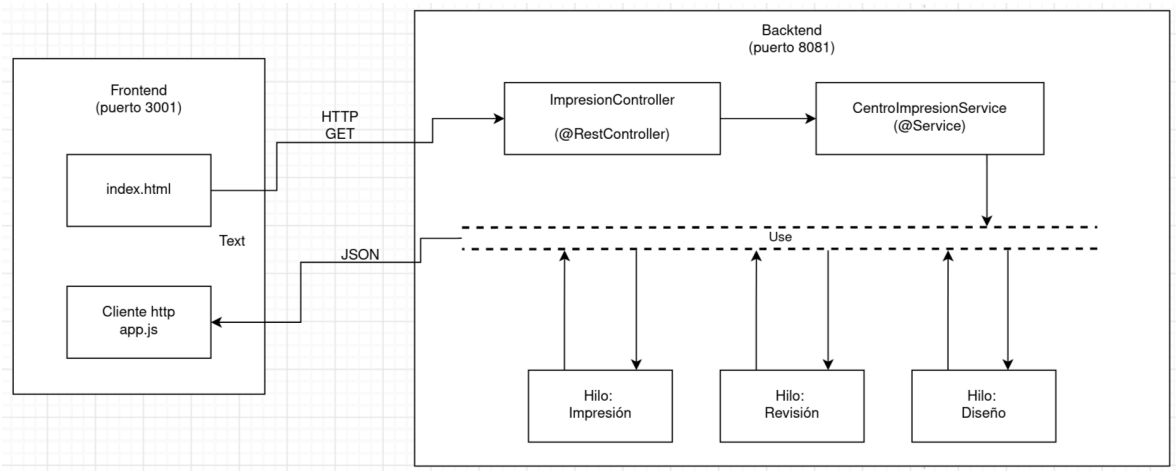
No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Estructura de archivos:

```
proyecto-distribuidos/
├── backend-impresion/
│   └── src/
│       └── main/
│           └── java/
│               └── com/
│                   └── practica/
│                       └── impresion/
│                           ├── ImpresionApplication .java
│                           ├── controller/
│                           │   └── ImpresionController .java
│                           └── service/
│                               └── CentroImpresionService .java
└── frontend-impresion/
    ├── index.html
    ├── style.css
```

└── app.js

Diagrama:



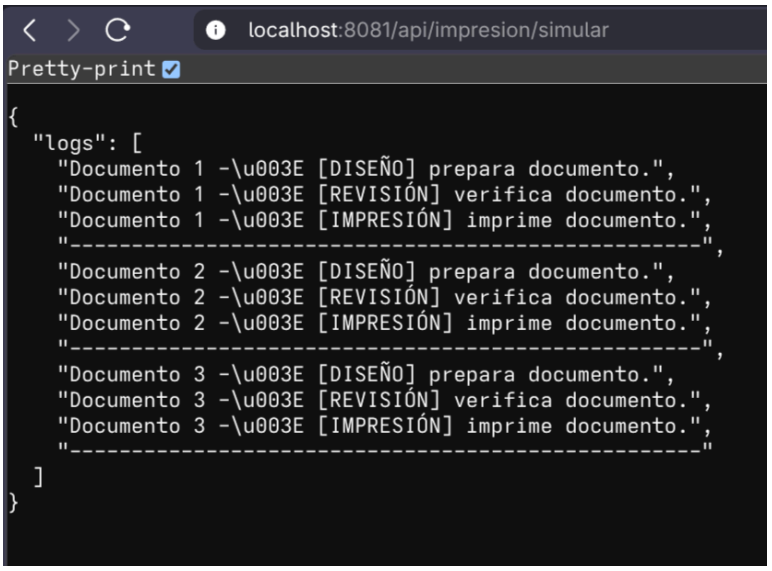
Para la realización del ejercicio 5 se mantiene un funcionamiento similar a microservicios comunicado a través del archivo ImpresionController.java.
La lógica se mantiene en el archivo CentroImpresionService.java el cual mantiene la estructura y algoritmo presentado por parte del docente.
Para la ejecución o levantamiento del servicio se presenta el uso de la herramienta Docker a través de archivos como pom.xml y Dockerfile tanto para el FrontEnd como para el Backend, todo esto orquestado por un único archivo en la raíz del proyecto: docker-compose.yml el cual se encarga de la creación de la imágenes de contenedores y su correspondiente funcionamiento.

1. Ejecución del archivo “docker-compose.yml” para la creación de contenedores y levantamiento de servicios.

```
…/APE3_avance_Isaac_Cartuche/ejercicio5 ❯ docker compose ps
WARN[0000] /home/isaaccartuche/Downloads/APE3_avance_Isaac_Cartuche/ejercicio5/docker-compose.yml: t
he attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusio
n
NAME                          IMAGE                              COMMAND                  SERVICE
       CREATED              STATUS              PORTS
impresion-backend-service     ejercicio5-backend-impresion       "java -jar app.jar"      backend-impres
ion    About a minute ago    Up About a minute   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
impresion-frontend-service    ejercicio5-frontend-impresion      "/docker-entrypoint.…"   frontend-impre
sion   About a minute ago    Up About a minute   0.0.0.0:3001->80/tcp, [::]:3001->80/tcp

…/APE3_avance_Isaac_Cartuche/ejercicio5 ❯
```

2. Respuesta correcta del Backend en navegador a través del API.



3. Ejecución correcta del Frontend.



7. Preguntas de Control:

- **¿Qué es una condición de carrera?**

  Es un problema de concurrencia que sucede cuando dos o más hilos - procesos tiene acceso a un mismo recurso, y al menos uno lo modifica, de modo que el resultado depende del orde o sincronización de sus acciones, por ejemplo si dos hilos incrementan al mismo tiempo una variable compartida, ambos pueden leer el valor inicial y escribir el mismo resultado, haciendo que se pierda uno de los incrementos, por ello para prevenir este problema se usan mecanismos de sincronización, como mutex, o semáforos [1], [2].
- **¿Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas**

  La diferencia es que un mutex garantiza la exclusión mutua mediante el bloqueo, mientras que el semáforo binario es un mecanismo de sincronización que sus valores pueden ser 0 y 1, el cual también se usa para coordinar hilos.
  Aunque ambos impiden que varios hilos entren simultáneamente , no deben considerarse intercambiables en todos los casos [1], [3]. Por ello no pueden usarse indistintamente sin analizar el objetivo de sincronización.
- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**

  El interbloqueo ocurre cuando dos o más hilos o procesos quedan bloqueados indefinidamente ya que esperan un recurso que otro está ocupando, por ello ninguno puede continuar ejecutándose [1]. [4].
  Las condiciones de Coffman para que se produzca un interbloqueo son las siguientes:
  1. **Exclusión mutua:** un recurso puede ser usado por un proceso a la vez
  2. **Retención y espera:** un proceso tiene uno o mas recursos mientras espera tener mas
  3. **No apropiación:** los recursos no pueden retirarse por la fuerza al proceso que los posee
  4. **Espera circular:** cuando hay una cadena cerrada de procesos, cada uno espera un recurso retenido por el siguiente.

  La condición que se elimina es la espera circular, para conseguirlo se establece un orden para obtener recursos y todos los hilos deben respetarlo [1], [4], [5].

8. Conclusiones:

- En el ejercicio 1 se implementó el semáforo contador como mecanismo de sincronización para la gestión de acceso a un recurso en este caso los 4 parqueaderos disponibles, gracias a la simulación se logró evidenciar consistencia de los datos en entornos multihilo, previniendo condiciones de carrera y coordinando las ráfagas de solicitudes simultáneas, los hilos que excedieron la disponibilidad del recurso fueron puestos en estado de espera bloqueada en orden FIFO, asegurando un acceso equitativo y evitando la inanición.
- En el ejercicio 3 entendí cómo se coordinan varios programas que usan un mismo recurso al mismo tiempo. En la fábrica, los productores fabrican

piezas, los consumidores las ensamblan y el almacén, con espacio para solo 5, es el lugar compartido. Lo resolvimos con dos semáforos y un mutex, que funciona como una llave para que un solo hilo a la vez toque la cola. En las pruebas vimos que cuando el almacén se llenaba esperaban los productores, y cuando se vaciaba esperaban los consumidores. Al final todo lo producido se consumió y el almacén nunca pasó de 5 piezas.
Lo que más me llamó la atención fue probar valores iniciales equivocados en los semáforos. Con espacios en 0 todos se quedaron esperando una señal que nunca llegaba y el sistema se trabó, y con elementos en 5 los consumidores intentaron sacar piezas que no existían y salieron errores. Entendí que esos valores no son un detalle, tienen que reflejar lo que realmente hay en el almacén.

- En el ejercicio 4 se coordinó el paso vehicular en un puente estrecho de un solo carril mediante un monitor protegido con un Mutex (ReentrantLock) y dos variables de condición (Condition para Norte y Sur). Con este esquema se garantizó de forma estricta la exclusión mutua de sentidos, impidiendo que vehículos en direcciones opuestas circularan simultáneamente y manteniendo la invariante !(Norte > 0 && Sur > 0) libre de colisiones. El aspecto más relevante de esta solución fue la resolución del reto de inanición (starvation) y monopolio de la vía: en lugar de permitir que un flujo ininterrumpido de autos en un sentido bloqueara indefinidamente al carril contrario, se implementó una política de equidad por lotes (Fair Batching) limitada a N vehículos consecutivos. Al alcanzar este límite y detectar tráfico en espera al otro extremo, el sistema suspende temporalmente los nuevos ingresos en await() y descongela a la cola opuesta con signalAll() cuando el puente se vacía, demostrando cómo la combinación adecuada de cerrojos justos y variables de condición permite construir sistemas concurrentes seguros y equitativos.
- En conjunto al ejercicio 5 se pudo identificar un escenario muy importante que suele pasar por desapercibido al confundir el control de ejecución de procesos y lo que es el acceso a memoria, se pudo evaluar algunos escenarios en donde teóricamente se realizaban procesos en secuencia, sin embargo sucede una situación de acceso a recursos (en este caso a un punto exacto de memoria) y si no se controla estos accesos el recurso por el que se compite puede dañarse, tener valores erróneos o inclusive errores por no encontrar el recurso que se necesita, para ello se presentó el uso de un controlador como mutex, permitiendo no solo controlar los procesos a nivel de orden y tiempo de ejecución, sino también controlar el acceso que tienen estos a recursos como la memoria y así mejorando la integridad de los datos que se desea mantener.

9. Evaluación

Criterio	Excelente	Bueno	Bajo
Funcionamiento (back-end, front-end) se evaluara criterios de Verificacion	Funciona completo (4)	Parcial (2)	No funciona (0)
Documento de practica (Enlace a git, metodología usada por el alumno, resultados, preguntas de control, bibliografía, conclusiones, video de explicación)	Funciona completo (3)	Parcial (2)	No funciona (0)
Defensa de practica	Todos los puntos (3)	Parcial (1)	No valido (0)

10. Declaratoria de IA

Para la presente práctica se hizo uso de herramientas de Inteligencia Artificial generativa (IA), específicamente de la herramienta "Google Gemini" en secciones como la identificación de la arquitectura tipo microservicios correspondiente a cada ejercicio al tratarse de una arquitectura nueva para el grupo de trabajo.
Se la utilizó también en parte del flujo de creación de código correspondiente al Frontend de cada ejercicio logrando una interfaz intuitiva y sencilla de manera más ágil.
Dentro de los promts utilizados están los siguientes:
- "Identifica, explica y propón una idea de organización de la arquitectura para cumplir un microservicio, la arquitectura debe ser funcionar para trabajar con Spring Boot y Java, además debe presentar una comunicación por API Rest, esto para el siguiente ejercicio: (pseudocódigo del ejercicio correspondiente)"
- "Genera un prototipo de frontend con fondo oscuro, colores simples y solo semánticos para genera énfasis en áreas como datos o grupos de tarjetas, el frontend debe corresponder al siguiente consumo de API: (API correspondiente a cada ejercicio)"

11. Bibliografía

Semana 3. Guaman, Jose. 2026

[1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ver. 1.10. Arpaci-Dusseau Books, 2023. [En línea]. Disponible: https://pages.cs.wisc.edu/~remzi/OSTEP/. Capítulos 28, 31 y 32

[2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, "Concurrency: An Introduction," en Operating Systems: Three Easy Pieces, cap. 26. [En línea]. Disponible: Sitio oficial del libro

[3] Linux man-pages project, "pthread_mutex_lock(3) — lock and unlock mutex," Linux man-pages. [En línea]. Disponible: Documentación técnica de pthread mutex

[4] W. Stallings, Operating Systems: Internals and Design Principles, 6.ª ed. Upper Saddle River, NJ, EE. UU.: Pearson Prentice Hall, 2009, cap. 6, "Deadlocks." [En línea]. Disponible: Texto de consulta sobre interbloqueos

[5] E. G. Coffman, Jr., M. J. Elphick, and A. Shoshani, "System Deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, Jun. 1971, doi: 10.1145/356586.356588

6. 10. Elaboración y Aprobación

Elaborado por	José Guamán Docente	Validar únicamente en Firmaec Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera Técnico Docente	Validar únicamente en Firmaec Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero Director de Carrera	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC